Remove Future-Related Explicit Specializations for Void

ISO/IEC JTC1 SC22 WG21
Document Number: P0241R0
Audience: Library Evolution Working Group
Matt Calabrese (metaprogrammingtheworld@gmail.com)
2016-02-11

Abstract

This paper uses the updated `void` type described in P0146R1[1] to simplify the specification of `std::promise`, `std::future`, and `std::shared_future` by removing the specializations for `void`. In order to ease transition of user code, some backwards compatibility is provided by way of deprecated functions that share an interface with those of the specializations of the current standard.

Motivation

In C++14 and prior C++ standards, `void` is a very unique type in the language and requires special handling in generic code. Perhaps the most prominent example of this is in the standard library regarding the `void` specializations for `std::promise`, `std::future`, and `std::shared_future`. The requirement of these specializations complicates standard library implementations, usage of `std::promise` and `std::future` in generic code, and even complicates the standard itself. With a `void` type that is an object type, as proposed in P0146R1[1], the need for these specializations goes away, and it would be beneficial both for the standard and for developers to remove these specializations.

Considerations

With the proposed `void` type, the simplest change to `std::promise` and `std::future` would be to simply remove the explicit specializations for `void`. This would work without doing anything else, however, there would be considerable breakage of existing code.

Interface Compatibility of `std::promise`

Perhaps the most obvious breaking change that comes from the removal of the `void` specializations is that the current function signature of the `set_value` member function of `std::promise<void>` differs from that of the generic template definition. More precisely, in the `void` specialization, `set_value` has no parameters (other than `this`) while in the generic definition, `set_value` is overloaded to take either an lvalue-reference-to-const or an rvalue-reference to the promise's value type. The implication of this is that users who dealt with an instance of `std::promise<void>` and invoked `set_value` in existing code would have their code now fail to compile. While this might be considered acceptable, it is suggested here that we can ease the transition by introducing a deprecated overload to the generic definition of `std::promise` that is only callable for `void` and that shares an interface with the existing `set_value` for `void`. The suggested overload to be added is as follows:

```
[[deprecated("Prefer invoking as set_value({}).")]]
void set_value()
{
  // Definition is only provided for example.
  set_value({});
}
```

In order to provide the deprecated `set_value` function for `std::promise<void>`, it may be defined in a CRTP base type of the `promise`. In addition to the above change, a similar update is suggested regarding `set_value_at_thread_exit`.

Handling of Qualified Void Types

Somewhat more implicit in the formal changes detailed below is the fact that, because `void` types are now handled via the generic template definition, cv-void types are directly usable with `std::promise` and `std::future`. In existing C++, such specializations would be ill-formed because the standard only provides explicit specializations for the unqualified `void` type. Similarly, reference-to-void types are also implicitly supported with the proposed `void` type, as the reference specializations already provided by the standard will work without modification for reference-to-void types.

Proposed Solution

Remove `void` specialization of `std::promise` from **§30.6.1 [futures.overview]** synopsis in paragraph 1:

> ~~`template <> class promise<void>;`~~

Remove `void` specialization of `std::future` from **§30.6.1 [futures.overview]** synopsis in paragraph 1:

> ~~`template <> class future<void>;`~~

Remove `void` specialization of `std::shared_future` from **§30.6.1 [futures.overview]** synopsis in paragraph 1:

~~template <> class shared_future<void>;~~

Modify **§30.6.5 [futures.promise]** paragraph 1:

The implementation shall provide the template promise and ~~two~~ the specialization~~s,~~ promise<R&> ~~and promise<void>~~. ~~These~~This specialization differs only in the argument type of the member functions set_value~~,~~ and set_value_at_thread_exit, as set out in ~~its~~ the descriptions, below.

Modify **§30.6.5 [futures.promise]** declarations and paragraph 15:

```
void promise::set_value(const R& r);
void promise::set_value(R&& r);
void promise<R&>::set_value(R& r);
void promise<void>::set_value();
```

15 *Effects:* atomically stores the value r in the shared state and makes that state ready (30.6.4). The fourth version is only present when the template argument of promise is void. It is functionally equivalent to calling the second version with a void argument. [*Note:* The fourth version is considered deprecated. Users should prefer to use either the first version or the second version instead. – *end note*]

Modify **§30.6.5 [futures.promise]** declarations and paragraph 21:

```
void promise::set_value_at_thread_exit(const R& r);
void promise::set_value_at_thread_exit(R&& r);
void promise<R&>::set_value_at_thread_exit(R& r);
void promise<void>::set_value_at_thread_exit();
```

21 *Effects:* Stores the value r in the shared state without making that state ready immediately. Schedules that state to be made ready when the current thread exits, after all objects of thread storage duration associated with the current thread have been destroyed. The fourth version is only present when the template argument of promise is void. It is functionally equivalent to calling the second version with a void argument. [*Note:* The fourth version is considered deprecated. Users should prefer to use either the first version or the second version instead. – *end note*]

Modify **§30.6.6 [futures.unique_future]** paragraph 4:

The implementation shall provide the template future and ~~two~~ the specialization~~s,~~ future<R&> ~~and future<void>~~. ~~These~~ This specialization differs only in the return type and return value of the member function get, as set out in its description, below.

Modify **§30.6.6 [futures.unique_future]** declarations and paragraphs 14 through 16:

```
R future::get();
R& future<R&>::get();
~~void future<void>::get();~~
```

14 Note: as described above, the template and its ~~two~~ required specialization~~s~~ differs only in the return type and return value of the member function get.

15 Effects: wait()s until the shared state is ready, then retrieves the value stored in the shared state.

16 Returns:

- future::get() returns the value v stored in the object's shared state as std::move(v).
- future<R&>::get() returns the reference stored as value in the object's shared state.
- ~~future<void>::get() returns nothing.~~

Modify **§30.6.7 [futures.shared_future]** paragraph 4:

The implementation shall provide the template shared_future and ~~two~~ the specialization~~s,~~ shared_future<R&> ~~and shared_future<void>~~. ~~These~~ This specialization differs only in the return type and return value of the member function get, as set out in its description, below.

Modify **§30.6.7 [futures.shared_future]** declarations and paragraphs 16 through 19:

```
const R& shared_future::get() const;
R& shared_future<R&>::get() const;
~~void shared_future<void>::get() const;~~
```

16 *Note:* as described above, the template and its ~~two~~ required specialization~~s~~ differs only in the return type and return value of the member function get.

17 *Note:* access to a value object stored in the shared state is unsynchronized, so programmers should apply only those operations on R that do not introduce a data race (1.10).

```
18 Effects: wait()s until the shared state is ready, then retrieves the value stored in the shared state.
19 Returns:
   – shared_future::get() returns a const reference to the value stored in the object's shared state.
     [ Note: Access through that reference after the shared state has been destroyed produces undefined
     behavior; this can be avoided by not storing the reference in any storage with a greater lifetime
     than the shared_future object that returned the reference. – end note ]
   – shared_future<R&>::get() returns the reference stored as value in the object's shared state.
   – shared_future<void>::get() returns nothing.
```

References

[1] Matt Calabrese: "Regular Void" http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0146r1.html